

Complete Technical Stack Guide

RAG-Based Clinical Decision Support / Triage Assistant

For every decision below: **what to use, how to use it, why, and what the real alternatives are** — so you're making informed choices, not just following a recipe.

1. What "type" of RAG should you actually build?

RAG isn't one architecture anymore — it's a spectrum, and picking the right point on it matters more than any single component choice.

The spectrum

- **Naive RAG:** chunk → embed → top-k cosine similarity → stuff into prompt. <cite index="18-1">A 2024 comprehensive RAG benchmark found naive RAG without advanced techniques answers only 44% of factual questions correctly, vs. 63% for state-of-the-art RAG and just 34% for LLMs with no retrieval at all.</cite> This is your floor, not your target.
- **Advanced RAG:** adds hybrid retrieval (dense + keyword) and reranking on top of naive RAG. <cite index="17-1">Nearly every public 2024–2025 benchmark shows BM25 plus dense embeddings, fused with Reciprocal Rank Fusion, beats either retrieval method alone, and a cross-encoder reranker on top adds another 5–15 points of MRR on hard queries.</cite> <cite index="21-1">This is the right production default for most applications — the sweet spot of cost vs. quality — and you should only reach for something more complex if this doesn't clear your accuracy bar.</cite>
- **Corrective RAG (CRAG) / Self-RAG:** <cite index="24-1">adds a lightweight evaluator that grades retrieved documents as correct, ambiguous, or incorrect, and can trigger a fallback (e.g., broader search or an explicit "insufficient evidence" response) when confidence is low, rather than forcing an answer from bad context.</cite> This maps *directly* onto your "abstention behavior" evaluation dimension from the dataset guide — it's not just an academic technique, it's literally what makes your system's abstention behavior real instead of aspirational.
- **Agentic RAG:** <cite index="21-1">replaces the linear pipeline with an agent that plans, retrieves, evaluates sufficiency, and re-retrieves in a loop rather than retrieving once.</cite> Highest quality ceiling, highest complexity and latency cost.
- **Graph RAG:** builds a knowledge graph over the corpus for multi-hop, relationship-heavy reasoning. Powerful but a genuinely large extra build (graph construction + graph-aware retrieval) — not recommended for your timeline unless your evaluation shows multi-hop reasoning is a major failure mode.

Decision framework (use this to justify your architecture in the paper)

<cite index="21-1">If the answer lives in a single chunk, naive RAG with a reranker is enough. If it needs facts from 2-3 documents, advanced RAG (hybrid + rerank + query transformation) is the right level. Only reach for graph RAG or full agentic loops if the query needs genuine multi-step, multi-hop reasoning across many documents.</cite> Clinical triage questions are mostly the "2-3 document" case (symptom → relevant guideline sections → recommendation) — **advanced RAG is your target architecture**, with a corrective/abstention layer added specifically because patient-safety framing demands it.

What to build, concretely

Recommended architecture (build in this order):

1. Naive RAG (week 2-3) — get the pipeline working end to end
2. Add hybrid retrieval + reranker (week 4-5) — this is your accuracy jump
3. Add a corrective/abstention layer (week 6-7) — this is your safety + novelty layer, and it's genuinely citable as "we adapted corrective RAG principles for clinical abstention behavior" in your Methods section

Why not go full agentic RAG: it roughly doubles latency and engineering complexity for query types (triage questions) that mostly don't need multi-step planning — the evaluation gain wouldn't justify the schedule risk for your timeline. Mention it explicitly as future work in your paper; that's a legitimate and common thing to do, and costs you nothing.

2. Embeddings — the most counter-intuitive choice in this whole stack

The obvious assumption ("use a medical-specific embedding model") is not clearly correct anymore — check this carefully before you default to it.

<cite index="10-1">In a recent evaluation of 14 embedding models, PubMedBERT — despite being trained specifically on medical data — scored below a plain BM25 keyword-search baseline on medical retrieval.</cite> A separate 2024 clinical-domain study reached the same conclusion: <cite index="14-1">generalist embedding models were found to be better at short-context clinical semantic search than specialized medical embedding models, with the specialized S-PubMedBERT model actually outperformed by fully generalist embedding models.</cite> A 2026 open-source benchmark confirms the pattern is architectural, not incidental: <cite index="13-1">PubMedBERT, fine-tuned specifically on PubMed title-abstract pairs, scores below the BM25 lexical baseline on the MedRAG benchmark, while modern general-purpose instruction-tuned retrievers outrank it by 0.22-0.25 points even on its own training domain — the reasoning being that a 2022-era 110M-parameter model without instruction-tuned retrieval objectives simply can't keep up with 2024-2026 generalist retrievers, domain match notwithstanding.</cite>

What this means for you: don't assume "medical embedding = better." Test empirically on your own corpus — this is also a legitimately interesting thing to report in your paper (a mini ablation: generalist vs. domain-specific embeddings on your specific clinical-guideline retrieval task).

Recommended options to actually test

Model	Type	Why consider it
BGE-M3 (open-source)	Generalist, supports dense + sparse + multi- vector retrieval in one model	<cite index="10-1">Best open-source all-rounder — gives you hybrid retrieval capability (dense + sparse) from a single model, which simplifies your pipeline.</cite> Free, self-hostable, no API cost — good fit for a student budget.
MedCPT (Google, open-source)	Medical-specific	<cite index="16-1">Pretrained on 255M query-article pairs from real PubMed search logs, achieving state-of-the-art zero-shot biomedical retrieval in its original benchmarks</cite> — worth including as your "domain-specific" comparison point precisely because the generalist-vs-specialist question is unsettled.
Voyage-3-large (API)	Generalist, commercial	<cite index="10-1">Best retrieval-focused API model, outperforming OpenAI's embeddings by 9.74% on retrieval benchmarks</cite> — good if you want a strong API-based baseline without self-hosting.
OpenAI text-embedding-3-large (API)	Generalist, commercial	<cite index="10-1">Best ecosystem integration and widest SDK support</cite> — the "safe default" if you're already using OpenAI for generation.
all-MiniLM-L6-v2 (open-source)	Lightweight generalist	<cite index="10-1">Best for edge/latency-critical deployments, zero API cost, minimal compute</cite> — not your primary choice, but useful for a fast local dev loop before running expensive full-corpus re-indexing.

My recommendation: run a small retrieval-quality ablation in week 2 (before committing) comparing **BGE-M3** (open-source, hybrid-native) against **MedCPT** (domain-specific) on a held-out sample of your own guideline corpus + MedQA queries. Whichever wins on your actual data is your answer — and reporting that comparison, with the counter-intuitive possible outcome, is a genuinely good paragraph in your paper. Default to **BGE-M3** if you don't have time to run the ablation — it's free, hybrid-capable, and the current evidence leans toward generalist models for this exact task type.

Don't trust benchmark leaderboards blindly — as one source puts it plainly: <cite index="10-1">MTEB is a starting point, not a verdict; domain specialists can

underperform on their own domains in some benchmarks, so test on your actual documents with your actual queries.</cite>

3. Chunking Strategy

<cite index="18-1">Hybrid retrieval combined with sentence-window chunking is the highest-ROI starting point for most production RAG systems</cite> — meaning your chunking choice interacts directly with your retrieval choice, not independently.

Recommended approach: structure-aware + sentence-window hybrid

1. **Primary split:** by document structure first (section headers like "Diagnosis," "First-line treatment," "Contraindications" — guideline documents are already well-structured, use that).
2. **Within each section:** sentence-window chunking — index individual sentences for precise retrieval, but expand to a surrounding window (2-3 sentences before/after) when passing context to the LLM. This gets you precise retrieval *and* enough context for coherent generation, which a flat fixed-size chunk doesn't.
3. **Size:** 256-512 tokens per chunk, ~10-15% overlap, as a starting point — validate against your own retrieval-precision metric rather than trusting this as gospel.
4. **Metadata on every chunk:** source guideline name, year, section type. This is what enables citation display ("grounded in NICE Guideline X, 2023") and lets you filter retrieval by section type later.

What NOT to do: naive fixed-size character splitting with no structure awareness — it's the fastest to implement but it's also exactly what "naive RAG" benchmarks badly, per the numbers above.

4. Hybrid Retrieval + Reranking

This is arguably your single highest-leverage architectural decision — it's the difference between the 44% and 63% accuracy numbers cited above.

Retrieval fusion

- **Dense retrieval:** your embedding model (BGE-M3 or chosen alternative) via cosine/dot-product similarity.
- **Sparse/keyword retrieval:** BM25 — critical for clinical text because exact-match terms (drug names, dosages, ICD codes) are common and dense embeddings can miss exact-term precision.
- **Fusion method:** Reciprocal Rank Fusion (RRF) — simple, well-established, combines both rankings without needing to learn fusion weights. BGE-M3 supports this natively if you use its built-in hybrid mode; otherwise implement RRF yourself (it's ~20 lines of code).

Reranking

- **Cross-encoder reranker on top of fused retrieval results:** re-scores your top ~20-50 candidates with a model that jointly encodes query+document (more accurate but slower than embedding similarity, which is why you only apply it to a shortlist, not the full corpus).
- **Options:** Cohere Rerank (API, easy), BGE-reranker or Jina-reranker (open-source, self-hostable).
- **Caveat worth knowing and reporting:** <cite index="18-1">the ARAGOG benchmark found Cohere Rerank showed no notable advantage over naive RAG in some tests — benchmark rerankers on your own corpus rather than assuming they help.</cite> This is exactly the kind of honest, evidence-based caveat that makes your Methods section more credible, not less.

5. Generation LLM (the model that actually writes the answer)

You already have RAG project experience with a specific API — **stick with what you know for week 1-2** to avoid burning setup time relearning a provider, then consider whether a swap is justified once your pipeline works end to end.

Option	Type	Tradeoff
Claude API (Anthropic)	Commercial API	Strong instruction-following and grounding behavior, good for a system that needs to say "insufficient evidence" rather than hallucinate — worth specifically testing your abstention behavior against this model's tendency to hedge appropriately.
GPT-4 class (OpenAI)	Commercial API	Widest tooling/ecosystem support if you're combining with OpenAI embeddings.
Open-source (Llama 3, BioMistral, Meditron)	Self-hosted	No per-token API cost, full control, but requires GPU access and more engineering time — only worth it if your institution has spare compute and you want a "fully open-source, deployable in low-resource settings" angle for your societal-impact framing.

Recommendation: use whichever commercial API you already have experience with for the core build (fastest path to a working system), and — if time allows in weeks 11+ — run one comparison against an open-source model specifically to strengthen your "deployable in low-resource clinics" narrative, since a fully open-source stack is a legitimately more compelling claim for that specific framing.

6. Orchestration Framework — or building it yourself

Option	What it gives you	Tradeoff
LlamaIndex	Strong, RAG-specific indexing/retrieval abstractions, good default document loaders	Best fit for RAG-first projects specifically (vs. LangChain's broader agent focus)
LangChain	Broadest ecosystem, good if you'll add agentic/corrective-RAG logic later	Heavier abstraction layer, can obscure exactly what's happening — a real cost when your paper needs to precisely describe your method
Haystack	Production-oriented, clean pipeline abstractions	Smaller community than the above two, still solid
Raw/custom (no framework)	Full control and transparency — every design choice is explicit and easy to describe precisely in your Methods section	More code to write yourself, no free document loaders/connectors

Recommendation: use **LlamaIndex** for the retrieval/indexing plumbing (loaders, chunking utilities, hybrid retrieval helpers) but **write your corrective/abstention logic and evaluation harness yourself, not via a framework's built-in agent abstractions**. Reason: for a research paper, you need to precisely describe and control exactly how your novel contribution (the abstention/corrective layer) works — burying it inside a framework's agent loop makes both your implementation and your Methods section muddier than it needs to be.

7. Vector Database

<cite index="26-1">A reasonable default path: start with ChromaDB to ship fast, and migrate to Qdrant when you need correct pre-filtering or hit scale.</cite>

Option	Best for	Key limitation
Chroma	Fast local prototyping, zero external service, week-1-2 dev loop	[26-1] Applies metadata filters <i>after</i> the similarity search (over-fetches to compensate), which degrades filtering correctness at scale — fine for your corpus size, but worth knowing why you might swap later. [26-1] Scale ceiling ~2-5M vectors.
Qdrant	Production-correctness, filtered hybrid search	[26-1] Written in Rust, applies filters <i>before</i> the ANN search — the technically correct behavior, which matters once you're filtering by guideline section/specialty metadata. [29-1] Widely considered the strongest default for low-latency filtered RAG at the 500K-50M vector scale. [26-1]
pgvector	If you're already running Postgres for your experiment-metadata DB (per your earlier ER diagram design)	[28-1] Keeps vectors in the same database as the rest of your app data — no second service to run or authenticate against — works well up to a few million vectors. [28-1]
Weaviate	Native hybrid search, GraphQL, multi-modal	[26-1] Schema-first approach adds friction while your embedding model is still changing during development, and self-hosted deployments need careful memory tuning. [26-1] Heavier than you need for this project.

Recommendation: Chroma for week 1-2 prototyping (zero setup friction, in-process Python), **migrate to Qdrant once your pipeline works** for the correct pre-filtered hybrid search you'll want when filtering retrieval by guideline metadata (section type, specialty, source). If you'd rather not run two systems at all and are already on Postgres for your metadata DB, **pgvector is a legitimate single-database simplification** — reasonable to pick for a research prototype where operational simplicity outweighs Qdrant's filtering-correctness edge at this scale.

8. Evaluation Framework (implementing the 4-part protocol from the dataset guide)

Tool	What it measures	Fit
RAGAS (open-source)	Faithfulness (groundedness), answer relevancy, context precision/recall — maps almost directly onto your accuracy/groundedness/retrieval-precision protocol	Best fit — purpose-built for exactly this evaluation shape, and widely cited in current RAG literature so your numbers are comparable to others'
TruLens	Similar triad of RAG metrics plus tracing/debugging tooling	Good alternative if you want more pipeline-debugging visibility during development
DeepEval	Broader LLM-eval framework, RAG metrics included	Good if you want unit-test-style eval integrated into CI, less RAG-specialized than RAGAS

Recommendation: RAGAS for your core faithfulness/relevancy/precision metrics (directly citable, widely used baseline), plus your own custom script for the **abstention-behavior** metric specifically, since that's your novel contribution and no off-the-shelf tool measures it out of the box.

9. Backend Stack

Given this project is ML-pipeline-heavy (not CRUD-heavy), lean Python-first rather than defaulting to your prior Node.js experience:

- **API framework: FastAPI** — async-native (matters for streaming LLM responses), auto-generated OpenAPI docs, native Pydantic validation which pairs cleanly with structured RAG outputs (citations, confidence scores). This is a deliberate deviation from your Node/Flask background because the ML-pipeline-heavy nature of this specific project favors Python's ecosystem (LlamaIndex, RAGAS, embedding libraries are all Python-first).
- **Metadata DB: PostgreSQL** — experiment configs, user sessions, query logs, evaluation results (same role as your ER diagram design in the FL doc, but simpler schema here: queries, retrieved-chunk references, generated answers, eval scores).
- **Auth: JWT** — you already have this experience, reuse it directly.

10. Frontend

- **React** (your existing skill) — a focused chat-style interface, not a full product build. Key screens: (1) query input, (2) streamed answer with inline citations linked to

source guideline chunks, (3) a visible "confidence/abstention" indicator when the system doesn't have grounded evidence — this last one is your most important UI element because it's the visual proof of your novel contribution to reviewers.

- **Don't over-invest in UI polish** — same guidance as the FL doc: this is a research demo, not a product; budget UI time accordingly (roughly 1-2 weeks total, not more).
-

11. Safety/Guardrails Layer

- Add an explicit, always-visible disclaimer that this is a research prototype, not a substitute for clinical judgment — state this in the proposal and the demo, not just buried in a README.
 - Consider a lightweight rule-based check (not a full framework like NeMo Guardrails, which is heavier than this project needs) that flags outputs mentioning medication dosages or high-risk terms for extra scrutiny/logging — a nice, easy-to-build safety-conscious touch that review panels notice.
-

12. Full Recommended Stack — Summary Table

Layer	Primary choice	Alternative(s)	Why primary
RAG architecture	Advanced RAG (hybrid + rerank) + corrective/abstention layer	Naive RAG (too weak) / Full agentic RAG (too complex for timeline)	Best accuracy-per-engineering-hour; abstention layer ties directly to your safety/novelty framing
Embeddings	BGE-M3 (test vs. MedCPT)	Voyage-3-large, OpenAI text-embedding-3-large	Free, hybrid-native, current evidence favors generalist models on this exact task
Chunking	Structure-aware + sentence-window, 256–512 tokens	Fixed-size naive chunking	Matches how guideline documents are actually organized; higher ROI per the cited benchmark
Retrieval fusion	Dense + BM25 via RRF	Dense-only	Directly addresses exact-match terms (drug names, dosages) that dense-only retrieval misses
Reranker	BGE-reranker or Cohere Rerank (benchmark both)	Skip reranking	Meaningful accuracy gain in most cases — but test on your corpus, don't assume
Generation LLM	Your existing API provider	Open-source (Llama/BioMistral) for low-resource-deployment angle	Fastest path to working system; swap-test later strengthens societal framing
Orchestration	LlamaIndex (retrieval plumbing) + custom code (abstention logic)	LangChain, Haystack, fully custom	Precise control over your novel contribution for the paper
Vector DB	Chroma (prototype) → Qdrant (production)	pgvector (if consolidating with Postgres)	Fast start, correct filtering once it matters

Layer	Primary choice	Alternative(s)	Why primary
Evaluation	RAGAS + custom abstention script	TruLens, DeepEval	Purpose-built, comparable to published RAG literature
Backend	FastAPI + PostgreSQL	Flask/Node (your prior stack)	Python-first ecosystem fit for ML-heavy pipeline
Frontend	React, focused chat UI	—	Matches your existing skill, no reason to change

13. Build Order (ties into the earlier week-by-week plan)

- Weeks 1-2:** data setup (per the dataset guide) + naive RAG with Chroma, default embeddings — get *something* end to end before optimizing anything.
- Weeks 3-4:** add hybrid retrieval (BM25+dense+RRF) and reranking; run your embedding-model ablation (BGE-M3 vs MedCPT) here.
- Weeks 5-6:** add the corrective/abstention layer — your core novel contribution.
- Weeks 7-8:** implement full RAGAS + custom eval harness; migrate to Qdrant if scaling past prototype.
- Weeks 9+:** proceed into the internal-review and paper-writing cadence from the earlier project doc.

Want me to turn this into the actual proposal document next, or start scaffolding the week-1 codebase (repo structure, dependencies, first pipeline script)?